A combinatorial Boolean-matrix-multiplication barrier for average-linkage hierarchical clustering

MohammadHossein Bateni, Laxman Dhulipala, Kishen N Gowda, D Ellis Hershkowitz, Rajesh Jayara

arxiv:2404.14730v2; CC BY 4.0

Editorial note. One named conditional lower-bound target is counted. Entire author paper and references retained; local input/include and author-supplied bbl are mechanically expanded. Unicode text-font wrapping is the only visual adjustment.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

Average linkage Hierarchical Agglomerative Clustering (HAC) is an extensively studied and applied method for hierarchical clustering. Recent applications to massive datasets have driven significant interest in near-linear-time and efficient parallel algorithms for average linkage HAC.

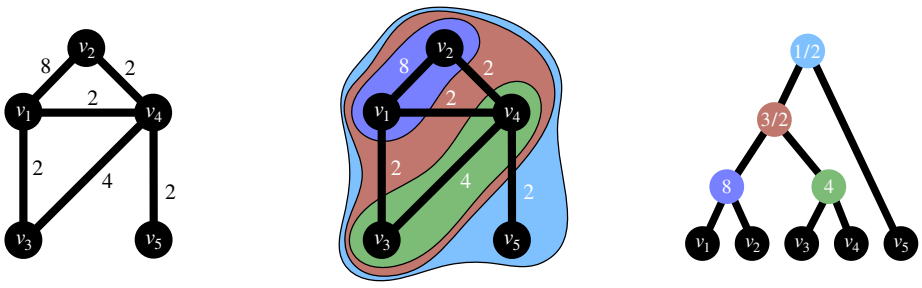
We provide hardness results that rule out such algorithms. On the sequential side, we establish a runtime lower bound of $n^{3/2-\epsilon}$ on n node graphs for sequential combinatorial algorithms under standard fine-grained complexity assumptions. This essentially matches the best-known running time for average linkage HAC. On the parallel side, we prove that average linkage HAC likely cannot be parallelized even on simple graphs by showing that it is CC-hard on trees of diameter 4. On the possibility side, we demonstrate that average linkage HAC can be efficiently parallelized (i.e., it is in NC) on paths and can be solved in near-linear time when the height of the output cluster hierarchy is small.

1 Introduction

Hierarchical clustering is a fundamental method for data analysis which organizes data points into a hierarchical structure so that similar points appear closer in the hierarchy. Unlike other common clustering methods, such as k-means, hierarchical clustering does not require the the number of clusters to be fixed ahead of time. This allows it to capture structures that are inherently hierarchical—such as phylogenies [18] and brain structure [10]. One of the most widely used and studied methods for hierarchical clustering is Hierarchical Agglomerative Clustering (HAC) [23, 25, 38]. HAC produces a hierarchy by first placing each point in its own cluster and then iteratively merging the two most similar clusters until all points are aggregated into a single cluster. The similarity of two clusters is given by a linkage function. HAC is included in many popular scientific computing libraries such as scikit-learn [36], SciPy [41], ALGLIB [37], Julia, R, MATLAB, Mathematica and many more [34, 35]. This cluster hierarchy is often equivalently understood as a binary tree—a.k.a. dendrogram—whose internal nodes correspond to cluster merges.

The proliferation of massive datasets with billions of points has driven the need for more efficient HAC algorithms that can overcome the inherent $\Theta(n^2)$ complexity required to read all pairwise distances [16, 17, 30]. Finer-grained running time bounds for HAC were recently obtained by assuming that only $m = o(n^2)$ pairs of points have nonzero similarity, and analyzing the running time as a function of both n and m. This is a natural assumption in practice, as in large datasets of billions of datapoints, typically a small fraction of pairs exhibit nonnegligible similarity. In this case, the input to HAC is an edge-weighted graph, where each vertex represents an input point and each edge weight specifies the similarity between its endpoints. This approach is convenient for large-scale applications since (1) very large clustering instances can be compactly represented as sparse weighted graphs and (2) the running time of HAC can be decoupled from the running time of nearest-neighbor search.

A particularly common linkage function for HAC is average linkage, which both optimizes reasonable global objectives [32] and exhibits good empirical performance [5, 11, 21, 24, 28, 30, 31, 33, 45]. Here, the similarity of two clusters is the average edge weight between them (non-present edges are treated as having weight 0). In other words, average linkage HAC repeatedly merges the two clusters with the highest average edge weight between them (see Figure 1 for an example).



(a) Input G. (b) HAC Output. (c) Dendrogram.

Figure 1: An example of average linkage HAC run on an input graph G. Edges labeled with weights. 1a gives G. 1b gives the cluster hierarchy output by HAC. 1c gives the corresponding dendrogram with internal nodes labeled with the weight of their corresponding merge.

A natural algorithmic question then is how quickly can we solve average linkage HAC on n node and m edge graphs? Recent work has provided a partial answer to this basic question in

sequential and parallel models of computation. In particular, [15] showed that average linkage HAC can be solved in $\tilde{O}(n\sqrt{m})$ time, thus providing a sub-quadratic time algorithm for sufficiently sparse graphs. A follow-up paper studied average linkage HAC in the parallel setting and showed that the problem is P-complete and so likely does not admit NC algorithms [16]. However, the P-completeness result of [16] holds for worst case graphs whereas typical applications of HAC are on highly structured graphs—namely those which are meant to capture relevant properties of an underlying metric—and so there is still hope for parallelizing average linkage HAC on more structured instances.

In fact, such structured instances of average linkage HAC are known to admit much faster algorithms in the sequential setting: the sequential algorithm of [15] implies that if the input graph is planar (or, more generally, minor-free) average linkage HAC can be solved in time $\tilde{O}(m)$. More generally, if each graph obtained by contracting all clusters at each step of average linkage HAC has $O(1)$ arboricity[1], then it is possible to solve average linkage HAC in time $\tilde{O}(m)$; it follows that average linkage HAC can be solved in sequential time $\tilde{O}(m)$ on trees or planar graphs. In light of these improved sequential results for highly structured graphs, it becomes natural to hope for efficient parallel algorithms on structured graphs such as low arboricity graphs or, even, just trees.

1.1 Our Contributions

In this work, we continue the line of work which studied the computational complexity of different variants of HAC [1, 15, 16, 20, 40] and perform a careful investigation into the complexity of average linkage HAC. In particular, we study HAC on n node and m edge graphs and investigate whether near-linear time algorithms, or more efficient parallel algorithms are possible, namely:

1. Near-Linear Time Algorithms: Can we improve over the best known $\tilde{O}(n\sqrt{m})$ upper bound for average linkage HAC and obtain near-linear time sequential algorithms?

2. NC Algorithms: are there $\mathsf{polylog}(n)$ depth parallel algorithms for average linkage HAC with $\mathsf{poly}(n)$ work for highly structured instances, e.g., trees, or minor-closed graphs?

We give both new lower bounds which (conditionally) rule out near-linear time and NC algorithms, and provide conditions under which these impossibility results can be bypassed.

First, we demonstrate that near-linear time algorithms are impossible under standard fine-grained complexity assumptions.

Theorem 1. If average linkage HAC can be solved by a combinatorial algorithm in $O(n^{3/2-\epsilon})$ time for any $\epsilon > 0$, then the Combinatorial Boolean Matrix Multiplication (Combinatorial BMM) Conjecture is false.

Our reduction also implies a second (weaker) conditional lower bound that also holds for non-combinatorial algorithms (e.g., algebraic algorithms) based on the running time of matrix multiplication. In particular, for two $n \times n$ binary matrices, it is well known that matrix multiplication can be solved in time $O(n^{\omega})$ where $2 \leq \omega < 2.3716$ [44]. In this setting, we obtain the following result:

Theorem 2. If average linkage HAC can be solved by an algorithm in $O(n^{\omega/2-\epsilon})$ time for some $\epsilon > 0$, then boolean matrix multiplication can be solved in $O(n^{\omega-\epsilon'})$ time for some $\epsilon' > 0$.

[1] A graph has arboricity at most α if all of its edges can be covered by at most α trees.

Notably, Theorem 1 shows that the prior running time of $\tilde{O}(n\sqrt{m})$ of [15] is optimal up to logarithmic factors under standard fine-grained complexity assumptions, at least for graphs consisting of $O(n)$ many edges. We obtain this conditional lower bound by showing that a carefully constructed instance of HAC can be used to solve the triangle detection problem, which is sub-cubically equivalent to Boolean Matrix Multiplication [43]. We obtain a bound of (essentially) $\Omega(n^{3/2})$ since our reduction incurs a quadratic time and space blowup when transforming an input triangle detection instance to an instance of average linkage HAC.

We next turn to the parallel setting. Here, we show that HAC—even on trees—is unlikely to admit efficient parallel algorithms. More formally, we show that average linkage HAC on low diameter trees is as hard as any problem in the complexity class Comparator Circuit (CC) [12, 39]. It is believed that CC is incomparable with NC and that CC-hardness is evidence that a problem is not parallelizable [12, 29].

Theorem 3. Average linkage HAC is CC-hard, even on trees of diameter 4.

We note that it is known that CC $\subseteq$ P and so the P-hardness of [16] already suggests the impossibility of efficient parallel algorithms on general graphs. However, our result suggests the impossibility of efficient parallel algorithms even on very simple graphs (trees of diameter 4). We obtain this result by reducing from the lexicographically first maximal matching (LFM Matching) problem and an intermediate problem which we call Adaptive Minimum, which captures some of what makes HAC intrinsically difficult to parallelize.

On the positive side, we demonstrate that average linkage HAC on path graphs is in NC, under the mild assumption that the aspect ratio is polynomial. While the class of path graphs is restrictive, even on paths average linkage is highly non-trivial and naively running HAC requires resolving chains of $\Omega(n)$ sequential dependencies. For example, consider a path of vertices $(v_1, v_2, \ldots, v_n)$ where the edge $\{v_i, v_{i+1}\}$ has weight $1 + i \cdot \epsilon$ for some small $\epsilon > 0$ and initially each vertex is in its own cluster. Initially, v_n's most similar neighbor is v_{n-1} and so v_n would like to merge with v_{n-1} but v_{n-1}'s most similar neighbor is v_{n-2} and so on. Thus, whether or not v_n gets to merge with v_{n-1} depends on the merge behavior of $\Theta(n)$ other clusters and so it is not at all clear that NC algorithms should be possible for this setting. Nonetheless, we show the following.

Theorem 4. Average linkage HAC on paths is in NC. In particular, there is an algorithm for average linkage HAC that runs in $O(\log^2 n \log\log n)$ depth with $O(n \log n \log\log n)$ work.

The above algorithm leverages the fact that in average linkage HAC the maximum edge similarity monotonically decreases. In particular, it works in $O(\log n)$ phases where each phase consists of merges of equal similarity up to constants. The goal then becomes to efficiently perform merges until every edge is no longer within a constant of the starting maximum similarity of the phase. The starting point of the algorithm is to observe that $\Omega(n)$ sequential dependencies of clusters of equal size can be resolved efficiently in parallel in a phase by noting that in this phase only the odd-indexed edges merge in the chain. Thus, each edge can decide if it is odd-indexed in parallel by, e.g., using prefix-sum, which is well known to be solvable in linear work in NC.

For chains with clusters of general weights, we decompose dependency chains into short ($(O(\log n)$-length) subchains where resolving dependencies within the subchain must be done sequentially but in the current phase each subchain's merge behavior only depends on whether or not its closest neighboring subchains merges into it or not. Thus, each subchain can compute its merge behavior for these two cases and then, similar to the equal weights setting, we propagate merge behavior across subchains efficiently in parallel.

To complement our sequential lower bound with a positive result, we demonstrate that it is possible to achieve near-linear running time, provided the dendrogram has low height. Thus, if the output dendrogram is a relatively balanced tree, then near-linear time algorithms are possible.

Theorem 5. There is an implementation of the nearest-neighbor chain algorithm for average linkage HAC that runs in $O(m \cdot h \log n)$ time where h is the height of the output dendrogram.

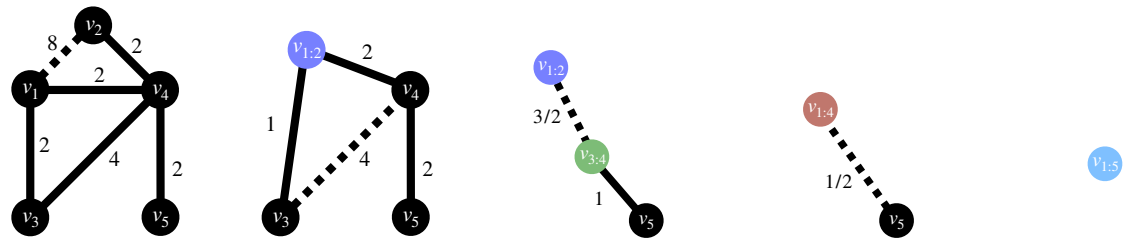
The above result is in fact obtained by a relatively simple (but to the best of our knowledge, new) analysis of existing classic HAC algorithms. In particular, we show that the nearest-neighbor chain [6, 22] and heap-based algorithms [28] for HAC, which were developed over 40 years ago achieve this bound.

2 Preliminaries

The input to the HAC algorithm is an undirected weighted graph $G = (V, E, w)$, where $w : V \times V \to \mathbb{R}_+ \cup \{0\}$ is a function assigning nonnegative weights to the edges. For convenience we assume $w(x, y) = 0$ when $xy \notin E$. The vanilla version of average linkage HAC is given as Algorithm 1. It starts by putting each vertex in a cluster of size 1 and then repeats the following step. While there is a pair of clusters of positive similarity, find two most similar clusters and merge them together, that is, replace them by their union. The similarity between two clusters is the total edge weight between them divided by the product of the cluster sizes. We refer to this version as the static graph version, since the graph is not changed throughout the run of the algorithm.

Throughout the paper we usually work with a different (equivalent) way of presenting the same algorithm which is given as Algorithm 2 (e.g., Algorithms 4 and 5). In this version we maintain a graph G whose vertices are clusters. The size of the vertex is the size of the cluster it represents. The normalized weight of an edge xy in G is $w(x, y)$ divided by the product of the sizes of x and y.

Whenever two clusters merge, their corresponding vertices are merged into one, i.e., the edge between them is contracted and the size of the new vertex is the sum of the sizes of the vertices that merged. In the following we sometimes say that a vertex x merges into vertex y. In this case we simply assume that the name of the resulting vertex is y and the size of y is increased by the size of x. See Figure 2.



(a) Input G. (b) 1 merge. (c) 2 merges. (d) 3 merges. (e) 4 merges.

Figure 2: An example of average linkage HAC run on an input graph G where we imagine we contract merged clusters. Intermediate vertices labeled with the vertices of G their corresponding cluster contains. Edges labeled with their weight and next merged edge is dashed.

The output of HAC is a dendrogram—a rooted binary tree representing the cluster merges performed by the algorithm. Every node of the dendrogram is a cluster built by the algorithm.

Input: $G = (V, E, w)$
Function `Similarity`(C_1, C_2, w):
  return $\sum_{x \in C_1, y \in C_2} w(x, y)/(|C_1| \cdot |C_2|)$
Function `HAC`(G):
  $\mathcal{C} \leftarrow$ clustering where each vertex of G is in a separate cluster
  while $\exists_{C_1, C_2 \in \mathcal{C}}$ s.t. $C_1 \neq C_2$ and `Similarity`$(C_1, C_2, w) > 0$ do
    $(C_1, C_2) = \arg\max_{(C_1, C_2) \in \mathcal{C} \times \mathcal{C}}$ `Similarity`(C_1, C_2, w)
    $\mathcal{C} := (\mathcal{C} \setminus \{C_1, C_2\}) \cup \{C_1 \cup C_2\}$.

Algorithm 1: Average linkage HAC—static graph version.

Input: $G = (V, E, w)$
Function `Similarity`(x, y, w, S):
  return $w(x, y)/(S(x) \cdot S(y))$
Function `HAC`(G):
  $S :=$ a function mapping each element of V to 1
  while $\exists_{xy \in E}$ s.t. `Similarity`$(x, y, w, S) > 0$ do
    $xy = \arg\max_{xy \in E}$ `Similarity`(x, y, w, S)
    Contract x with y in G creating a vertex z. The parallel edges that are created are merged into a single edge whose weight is the sum of the merged edge weights. Any resulting self-loops are removed.
    Set $S(z) := S(x) + S(y)$

Algorithm 2: Average linkage HAC—graph contraction version.

There are exactly $|V|$ leaves corresponding to the single-element clusters that are formed in the beginning of the algorithm. Whenever two clusters C_1 and C_2 are merged, we add to the dendrogram a new node $C_1 \cup C_2$ whose children are C_1 and C_2. See Figure 1c for the dendrogram of Figure 2.

We use the classic multithreaded model [4, 8, 9] (formally, the MP-RAM [8]) to analyze the parallel algorithms. We assume a set of threads that share the memory. Each thread acts like a sequential RAM plus a `fork` instruction that forks two new child threads. When a thread performs a fork, the two child threads can both start by running their next instructions, and the original thread is suspended until both children terminate. A computation starts with a single root thread and finishes when that root thread finishes. A parallel for-loop can be viewed as executing `fork`s for a logarithmic number of levels. A computation can thus be viewed as a DAG (directed acyclic graph). We say the work is the total number of operations in this DAG and span (depth) is equal to the longest path in the DAG. We note that computations in this model can be cross-simulated in standard variants of the PRAM model in the same work (asymptotically), and losing at most a single logarithmic factor in the depth [8].

3 An $\Omega(n^{3/2-\epsilon})$ Conditional Lower Bound for Average Linkage HAC

In this section, we show an $\Omega(n^{3/2-\epsilon})$ conditional lower bound on the time required to solve average linkage HAC on general weighted graphs. Specifically, we show this lower bound assuming the Combinatorial Boolean Matrix Multiplication (BMM) conjecture, a central conjecture in fine-grained complexity about the time required to multiply two $n \times n$ boolean matrices [2, 43].

Conjecture 1 (Combinatorial BMM). Combinatorial algorithms cannot solve Boolean Matrix Multiplication in time $O(n^{3-\epsilon})$ for $\epsilon > 0$.

We refer to [2] for an in-depth discussion of the somewhat informal notion of "combinatorial" algorithms and more on Conjecture 1 and its history.

In this work we will make use of an equivalent characterization of the BMM conjecture due to [43]. Specifically, [43] shows that the BMM problem is sub-cubically equivalent to the Triangle Detection problem: the problem of deciding whether or not an input graph G contains a triangle (i.e., cycle of 3 vertices). The following summarizes this result.

Theorem 6 (Theorem 1.3 of [43]). Combinatorial algorithms cannot solve Triangle Detection in time $O(n^{3-\epsilon})$ for $\epsilon > 0$ unless Conjecture 1 is false.

Thus, we give a reduction from Triangle Detection to average linkage HAC. Our reduction will quadratically increase the number of vertices of the input Triangle Detection instance, and therefore give an $\Omega(n^{3/2-\epsilon})$ lower bound for average linkage HAC. In the rest of this section, we show the following quadratic-blowup reduction from Triangle Detection to average linkage HAC.

Theorem 7. Given a Triangle Detection instance on graph G with t vertices and m edges, there is a reduction that runs in $O(t^2)$ time and constructs an instance of average linkage HAC on graph G' with $t + t^2$ vertices and $t^2 + m$ edges. Furthermore, given the sequence of merges performed by average linkage HAC on G', we can solve Triangle Detection on G in time $O(t^2)$.

As a corollary of this reduction and Theorem 6, we obtain the following conditional lower-bound on the running time of HAC.

Theorem 1. If average linkage HAC can be solved by a combinatorial algorithm in $O(n^{3/2-\epsilon})$ time for any $\epsilon > 0$, then the Combinatorial Boolean Matrix Multiplication (Combinatorial BMM) Conjecture is false.

As a second corollary, we obtain a conditional lower-bound in terms of the optimal running time of matrix multiplication for two $n \times n$ binary matrices. Matrix multiplication can be solved in time $O(n^\omega)$ where $2 \leq \omega < 2.3716$ [44]. An extensive line of research on matrix multiplication over the past thirty years has only improved ω from 2.376 to 2.3716, with the current state-of-the-art being due to a very recent result of Williams et al. [44] (for a subset of the historical advances in this area see, e.g., [3, 13, 27, 42]). The fastest known algorithm for triangle detection works by simply reducing the problem to matrix multiplication and therefore runs in $O(n^\omega)$ time. Surprisingly, despite triangle detection only returning a single bit (whether a triangle exists or not in G), the problem can be used to give a sub-cubic reduction for boolean matrix multiplication (where the output is n^2 bits). In particular, an algorithm for triangle detection running in time $O(n^{3-\delta})$ for some $\delta > 0$ yields an algorithm for matrix multiplication in time $O(n^{3-\delta/3})$ [43]. Using this fact, we can derive a conditional lower bound based on the value of ω.

Theorem 2. If average linkage HAC can be solved by an algorithm in $O(n^{\omega/2-\epsilon})$ time for some $\epsilon > 0$, then boolean matrix multiplication can be solved in $O(n^{\omega-\epsilon'})$ time for some $\epsilon' > 0$.

An interesting open question is whether there are faster non-combinatorial algorithms that can leverage fast matrix multiplication or Strassen-like techniques and improve over the $\Omega(n^{3/2-\epsilon})$ barrier for combinatorial algorithms for average linkage HAC.

3.1 Reduction

We now prove Theorem 7 by giving a quadratic-time reduction from triangle detection to average linkage HAC. The reduction is loosely inspired by a recent lower-bound result for multidimensional range queries [26]. The input to the reduction is an unweighted graph G on t vertices with m edges; the problem is to detect whether G has a triangle. To do this, we will construct a HAC instance on an edge-weighted graph G' with $t + t^2$ vertices and $t^2 + m$ edges. We will show that the specific way in which an exact HAC algorithm merges the edges in this instance reveals whether or not G has a triangle.

Constructing G' Let $N_G(v)$ denote the neighbors of a vertex $v \in G$ (note that $v \notin N_G(v)$). We define G' as follows. We start by adding all vertices and edges from G, that is the t vertices $v_1, \ldots, v_t$ from G, including all of their incident edges $N_G(v_i)$. We call these the core vertices. The initail weight of the edges between any two core vertices is set to 1.

In addition to the core vertices, we add an additional t^2 leaf vertices that we connect to the core vertices with specific edge weights. We add the t^2 leaf vertices over a sequence of t rounds where the i-th round connects one new leaf vertex to every core vertex. The weights to the newly added leaves depend on the neighbors of the node v_i in the original graph G, and are set as follows:

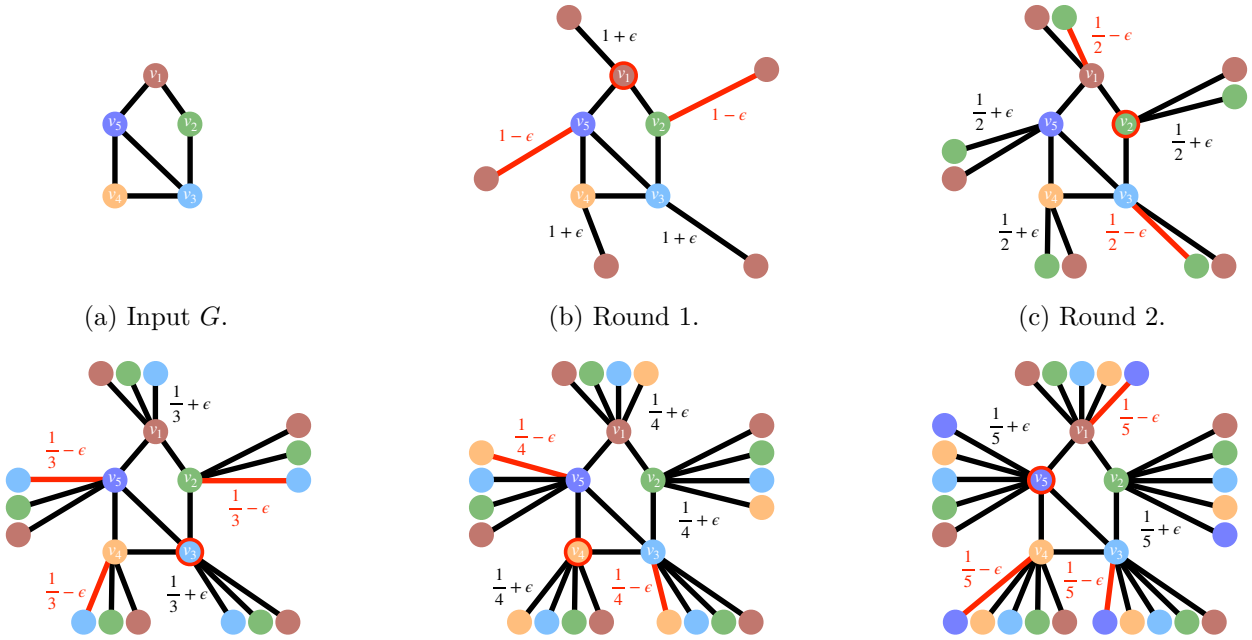
(1) A core vertex v_j is connected to its new leaf with an edge of weight $(1/i) - \epsilon$ if $v_j \in N_G(v_i)$.

(2) A core vertex v_j is connected to its new leaf with an edge of weight $(1/i) + \epsilon$ if $v_j \notin N_G(v_i)$.

See Figure 3 for an illustration of our reduction.

Running HAC on G' Having defined G', let us consider the merges that the exact HAC algorithm will make on this instance. In this section, for succinctness we use weight to refer to the normalized (i.e., average linkage) weight. HAC will begin by merging the maximum weight edges. The maximum weight initially depends on the structure of $N_G(v_1)$. First, all core vertices that are not in $N_G(v_1)$ will merge with their round 1 leaves (since these edges have weight $1 + \epsilon$) increasing their cluster size to 2. This leaves all core vertices in $N_G(v_1)$. Any edge in G' between two such core vertices will have weight 1, and will be merged next. Crucially, any merge in this round with a weight of 1 indicates a triangle incident on v_1 since the two core vertex endpoints of the edge must be contained in $N_G(v_1)$, hence connected by edges to v_1. If no edges of weight 1 merge, the remaining leaves that we added to core vertices in $N_G(v_1)$ merge into their neighboring core vertex.

Assuming we did not merge any weight 1 edges in the first round, at this point the cluster size of each core vertex is 2, and so the weight of any edge originally in G (between two core vertices) will be $1/4$. The edge weights to leaves in round 2 will be $((1/2) \pm \epsilon)/2 = 1/4 \pm (\epsilon/2)$, which is larger than $1/4$ for edges of Type (2). Therefore, the same argument for how edges merge in round 1 can be inductively applied to the next round. The edge weights in round i for edges between core



(a) Input G. (b) Round 1. (c) Round 2.

(d) Round 3. (e) Round 4. (f) Round 5.

Figure 3: Our triangle detection reduction where we compute G' from G by adding $t = 5$ nodes over t rounds. 3f gives G'. Each node labeled according to its round and corresponding vertex in G. Edges labelled with their weight in the round they are added (edges of G have weight 1). For the ith round we highlight in red v_i and the edges added with weight $1/i - \epsilon$.

vertices will be $1/i^2$, and by the same argument as before, the Type (2) (Type (1)) edges will be larger (smaller) by ϵ/i. As a result of how an exact average linkage HAC will merge the edges of G', we obtain the following lemma:

Lemma 1. Consider the sequence of merges performed by the HAC algorithm on G'. If the merge sequence consists of t^2 merges, which first merge all leaf vertices, and only then makes merges between core vertices, then G does not contain any triangles. If the merge sequence merges any edge between two core vertices in the first t^2 merges, then G contains a triangle.

Completing the Reduction We will now complete the proof of Theorem 7. Suppose we are given an instance of Triangle Detection on n vertices. Conjecture 1 implies that this instance cannot be solved by combinatorial algorithms in $O(n^{3-\epsilon})$ time for any $\epsilon > 0$.

Let the time complexity of HAC on a graph G with n vertices and m edges be $T_{\mathsf{HAC}}(n, m)$. Suppose HAC can be solved combinatorially in $O(n^{3/2-\epsilon})$ time. Given a Triangle Detection instance on n vertices we create a graph G' with $O(n^2)$ vertices and $O(n^2)$ edges, and run HAC on G'. The running time of the reduction is $O(n^2)$, and the running time of HAC on G' is $O((n^{2\cdot(3/2-\epsilon)}) = O(n^{3-2\epsilon})$, which will falsify Conjecture 1 by Theorem 6. Thus, conditional on Conjecture 1, there is no algorithm for HAC running in time $T_{\mathsf{HAC}}(n, m) = O(n^{3/2-\epsilon})$ for any constant $\epsilon > 0$, completing the proof of Theorems 7 and 1. The same argument, under the assumption that triangle detection cannot be solved in $O(n^{\omega-\epsilon})$ time for any constant $\epsilon > 0$ implies Theorem 2.

4 Average Linkage HAC is Hard to Parallelize Even on Trees

In this section we prove that average linkage HAC is likely hard to parallelize by showing it is CC-hard even on low depth trees. We begin with some preliminaries. The formal definition of CC-hardness we will use is as follows.

Definition 1 (CC-Hard). A problem is CC-hard if all problems of CC are logspace-reducible to it.

For our purposes we will not need to define the class CC. Rather, we only need the above definition of CC-hardness and a single CC-hard problem, LFM Matching. Recall that a matching of a graph $G = (V, E)$ is a subset of edges $M \subseteq E$ if each vertex is incident to at most one edge of M. A matching is said to be maximal if each $e = \{u, v\} \notin M$ satisfies the property that either u or v is incident to an edge of M. The greedy algorithm for maximal matching initializes M as $\emptyset$ and then simply iterates over the edges of E in some order and adds the current edge e to M if the result of doing so is a matching.

Problem 1 (LFM Matching). An instance of lexicographically first maximal matching (LFM Matching) consists of a bipartite graph $G = (V = L \sqcup R, E)$ with vertices ordered as $L = (l_0, l_1, \ldots, l_{n-1})$ and $R = (r_0, r_1, \ldots, r_{n-1})$. The lexicographically first maximal matching is the matching obtained by running the greedy algorithm for maximal matching on edges ordered first by their endpoint in L and then by their endpoint in R. That is, in this ordering $e = \{l_i, r_j\}$ precedes $e' = \{l_{i'}, r_{j'}\}$ iff (1) $i < i'$ or (2) $i = i'$ and $j < j'$. Our goal is to decide if a designated input edge is in the LFM Matching.

The following summarizes known hardness of LFM Matching.

Theorem 8 ([29, 39]). *LFM Matching* is CC-hard.

Next, we introduce the search variant of the HAC problem whose CC-hardness we will prove.

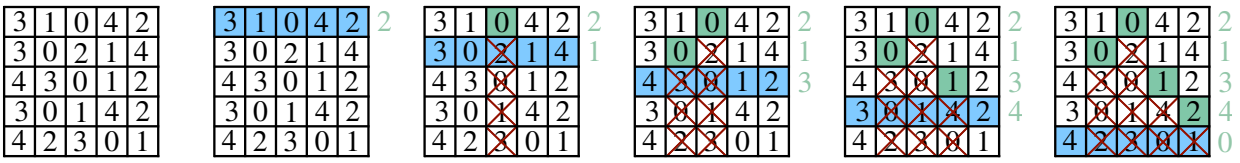
Problem 2 (Average Linkage HAC). An instance of *Average Linkage HAC* consists an undirected graph $G = (V, E)$, along with edge weights $w : E \to \mathbb{R}_{\geq 0}$. Consider the sequence $(C_1, C_1'), (C_2, C_2'), \ldots$ of cluster merges produced by the procedure `HAC(`G`)` from Algorithm 1. Given any pair of vertices $u, v \in G$, the goal of the *Average Linkage HAC* problem is to output the index i such that u, v first merge together at step i, namely $u \in C_i$ and $v \in C_i'$ (or $u \in C_i'$ and $v \in C_i$).

To prove the hardness of Average Linkage HAC, we will first prove the hardness of an intermediate problem, called Adaptive Minimum. The construction of the Adaptive Minimum problem will be more amenable to our reductions, and therefore simplify the following exposition. See Figure 4 for an illustration of Adaptive Minimum.

Problem 3 (Adaptive Minimum). An instance of *Adaptive Minimum* consists of a (0-based indexed) $n \times n$ matrix A where each row contains a permutation of $\{0, \ldots, n-1\}$ and some index $x \in [0, n)$. The goal is to simulate the following algorithm. Start with $I = \{0, \ldots, n-1\}$ and execute the following steps for $i = 0, \ldots, x$:

1. Let $k_i = \arg\min_{j \in I} A[i, j]$.
2. Set $I := I \setminus \{k_i\}$.

Our goal is to compute k_x.



(a) Input A. (b) Step 1. (c) Step 2. (d) Step 3. (e) Step 4. (f) Step 5.

Figure 4: Adaptive Minimum on matrix A. The row considered in each step is shown in blue. k_i for the i-th row written to the right of A in green with witnessing entry of A also in green. Indices removed from I in relevant rows crossed out in red.

Observe that both problems 2 and 3 are defined as having an algorithm output an index $i \in \{1, 2, \dots\}$. Thus, these can be considered search problems instead of decision problems. We choose to work with the search versions of these problems for simplicity of our reductions, however, our reduction naturally extends to the decision variants (e.g., where the algorithm is given $u, v \in V$ and an index i and asked if u, v merge on step i).

We first prove the CC-hardness of this intermediate problem. See Figure 5 for an illustration of our reduction from LFM Matching to Adaptive Minimum.

Lemma 2. *Adaptive Minimum* is CC-hard.

Proof. By Theorem 8 and the definition of CC-hardness (Definition 1), it suffices to argue that LFM Matching (Problem 1) is logspace reducible to Adaptive Minimum (Problem 3). We begin by describing our reduction and then observe that it only requires logarithmic space. The basic idea of the reduction is to associate with each vertex on the left side of our instance of LFM Matching a row of the matrix of Adaptive Minimum and each vertex on the right side of LFM Matching a column of the matrix of Adaptive Minimum.

More formally, consider an instance of LFM Matching on graph $G = (V = L \sqcup R, E)$ where our goal is to decide if a given edge e is in the LFM matching. We consider the following instance of Adaptive Minimum to solve this on a $2n \times 2n$ size matrix A. We will refer to an index i as a dummy index if $i \geq n$. We will construct the ith row of A to correspond to a permutation that first gives the indices of all neighbors of l_i in R sorted according to the ordering of R then gives all dummy indices then gives the indices of non-neighbors of l_i in R.

More formally, for a vertex $l_i \in L$, we let $R_i \subseteq R$ be the vertices of G connected to l_i and we let $\bar{R}_i = R \setminus R_i$ be all vertices not adjacent to l_i. Furthermore, for $r_j \in R_i$, we let $\mathrm{ind}(r_j) \in [|R_i|]$ be the index of r_j in R_i when we sort the vertices of R_i according to the ordering on R. Note that, in general, it need not be the case that $\mathrm{ind}(r_j) = j$. Symmetrically, for a vertex $r_j \in \bar{R}_i$, we let $\mathrm{ind}(r_j) \in [|\bar{R}_i|]$ be the index of r_j when we sort $\bar{R}_i$ by the ordering on R. Then, we define A as follows.

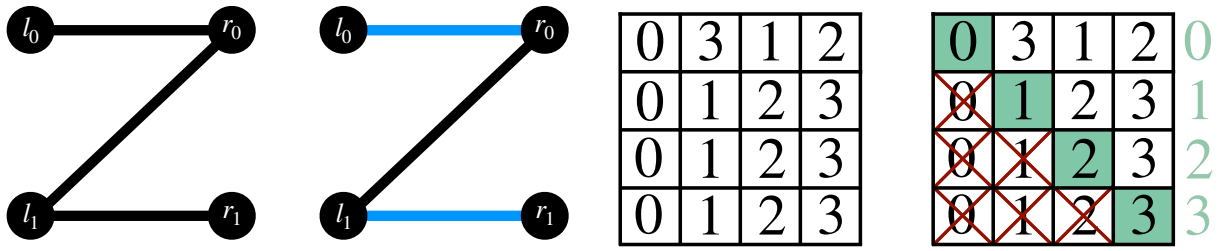
$$A[i, j] := \begin{cases} \mathrm{ind}(r_j) & \text{if } j < n \text{ and } r_j \in R_i \\ j - n + |R_i| & \text{if } j \geq n \\ \mathrm{ind}(r_j) + n + |R_i| & \text{if } j < n \text{ and } r_j \notin R_i \end{cases}$$

Lastly, let x (the index for which we would like to compute k_x in our instance of Adaptive Minimum) be the index of the endpoint of e in L. Once Adaptive Minimum computes k_x, we verify

whether or not it corresponds to the endpoint of e in R to determine the final output of the LFM Matching instance. Again, see Figure 5.

We now argue correctness of the reduction. A straightforward proof by induction on i demonstrates that at the beginning of the ith round of the Adaptive Minimum algorithm we have that I consists of at least $n - i$ dummy indices and $j < n$ is not in I only if r_j is in the LFM Matching and is matched to some $l_{i'}$ for $i' < i$. It follows that $e = (l_i, r_j)$ is in the LFM Matching iff $k_i = j$, showing correctness of our reduction.

It remains to show that the above reduction can be done with logspace. In order to do so, we must argue that $A[i, j]$ can be computed with logspace for every i and j. Doing so is trivial given the above definition of $A[i, j]$. □



(a) LFM Matching. (b) Solution. (c) Adaptive Minimum. (d) Solution.

Figure 5: Reduction from LFM Matching to Adaptive Minimum. 5a gives the LFM Matching instance and 5b its solution. 5c gives the Adaptive Minimum instance from the reduction and 5d its solution.

Concluding, we use the CC-hardness of Adaptive Minimum to prove the CC-hardness of Average Linkage HAC.

Theorem 3. Average linkage HAC is CC-hard, even on trees of diameter 4.

Proof. Our reduction shows how to reduce an instance of Adaptive Minimum (Problem 3) of size n to an instance of average linkage HAC on a tree. We build a rooted tree, in which each root-to-leaf path has length 2 (i.e., the tree has depth 2). We call the neighbors of the root internal nodes. Observe that each node is either the root, an internal node or a leaf. The fact that the tree is rooted is only for the convenience of the description. In the construction, we will begin by assigning each node an initial size (see the definition of size in Section 2) which is possibly larger than 1 (but at most $\mathsf{poly}(n)$). We will later show how to remove these variable sizes, and reduce to the case where all nodes have initial size 1 (as in the original definition of HAC).

The basic idea of our construction is as follows. Our HAC instance will consist of a rooted tree where each child of the root corresponds to a column of A in our Adaptive Minimum instance. HAC merges will then happen in phases where each phase corresponds to a row of A. In a given phase, exactly one internal node will merge with the root which will correspond to this internal node's column being minimum for the corresponding row of A. In order to guarantee this, each child of the root will have its own carefully selected children such that merging with these children guarantees the desired behavior in every phase.

More formally, the tree is constructed as follows. The root r of the tree has initial size n^8. It has n children, each of initial size n^4—we denote them by $v_0, \ldots, v_{n-1}$. The root is connected to

its children using edges of weight 1, i.e., $w(r, v_i) = 1$ for all $i = 0, 1, \ldots, n-1$ (thus, the normalized weight of the edges $\{rv_i\}_{i=0}^{n-1}$ are each $\frac{1}{n^{12}}$ at the start). Each internal node has $n(n+1)$ children (leaf nodes) grouped into n groups of $n+1$ leaves each. We write $C_{i,j} = \{v_{i,j,0}, v_{i,j,1}, \ldots, v_{i,j,n}\}$ to denote the j-th group of children of the i-th internal vertex. All the leaves $v_{i,j,k}$ have initial size 1. Thus, the vertices in the full graph in our construction consists of the root r, internal nodes $\{v_0, v_1, \ldots, v_{n-1}\}$ and leaves $\cup_{i=0}^{n-1} \cup_{j=0}^{n-1} \cup_{k=0}^{n} \{v_{i,j,k}\}$

Let $r_i = n^8 + i \cdot n^4$ (for $0 \leq i < n$). For each pair of an internal node and each of its groups there are only two distinct edge weights for edges between the internal node and the leaves in the group. Specifically, for an internal node v_j and group $C_{j,i}$ of its children we have $A[i,j] + 1$ edges of weight $\frac{1}{r_i - 1}$ and $n - A[i,j]$ edges of weight $\frac{1}{r_i + i \cdot n^3}$. Specifically, we set

$$w(v_j, v_{j,i,k}) = \begin{cases} \frac{1}{r_i - 1} & \text{if } 0 \leq k \leq A[i,j] \\ \frac{1}{r_i + i \cdot n^3} & \text{if } A[i,j] < k \leq n \end{cases}$$

We call the two weights high-weight and low-weight edges, respectively. Note that their normalized weights are $\frac{1}{n^4(r_i - 1)}$ and $\frac{1}{n^4(r_i + i \cdot n^3)}$ respectively. Observe that the setting of high-weight and low-weight edges is independent of the internal node v_j, although the number of high versus low-weight edges depends on $A[i,j]$. Moreover, note that even low-weight edges of any group $C_{j,i}$ have higher weights than high-weight edges of group $C_{j,i+1}$:

$$\begin{aligned} \frac{1}{n^4(r_i + i \cdot n^3)} &> \frac{1}{n^4(r_{i+1} + (i+1) \cdot n^3)} \iff \\ \frac{1}{n^8 + i \cdot n^4 + i \cdot n^3} &> \frac{1}{n^8 + (i+1)n^4 - 1} \iff \\ i \cdot n^3 &< n^4 - 1. \end{aligned}$$

which follows from the fact that $i \leq n - 1$. We now demonstrate that the average linkage HAC on this instance works in n phases numbered from 0 to $n - 1$, where in each phase i,

- $n - 1$ internal nodes contract all of their incident group i edges, and
- one internal node contracts all of its high-weight edges to group i, after which it merges with the root.

Because of the internal node merging with the root, the root has incident leaves, but they are connected with edges of (normalized) edge weights $\leq \frac{1}{n^{15}}$ and so they will be irrelevant until all phases have been completed. We show that if we denote by k_i the index of the internal node which merges with the root in phase i, the sequence $k_0, \ldots, k_{n-1}$ is a correct solution to the Adaptive Minimum problem.

In order to analyze the algorithm, we prove the following claim. For convenience, let us define $w_i = n^4 + i \cdot (n+1)$.

Claim 1. In the beginning of phase i, the graph is as follows:

1. The size of the root node is $r_i + i^2 \cdot \Delta_i$ for some $\Delta_i \in [0, n+1]$.
2. Exactly i internal nodes have been merged with the root, and the corresponding values $k_0, \ldots, k_{i-1}$ have been computed correctly.

3. The size of each of the $n-i$ remaining internal nodes is w_i.

4. For all remaining internal nodes, all leaves in groups $0,\ldots,i-1$ have been merged into their parents, and no leaves in groups $i,\ldots,n-1$ have been merged.

5. The root may have incident leaves (resulting from internal nodes contracting into it) connected to the root with edges of normalized weights $\leq \frac{1}{n^{15}}$.

Proof. We prove the above claim using induction on i. The base case of $i=0$ follows directly from how the tree is constructed.

We now simulate a single phase. The edges between the root and the internal nodes have (normalized) weights $\frac{1}{w_i(r_i+i^2\cdot\Delta_i)} > \frac{1}{n^{15}}$. Hence, the additional leaf nodes incident to the root (see Item 5 of the Claim) are irrelevant. Thus, the highest weight edge in the graph is surely incident to one of the internal nodes. Observe that the relative order of edge weights between an internal node v and its children does not change as the leaves are merged into v. Therefore, given that groups $0,\ldots,i-1$ do not exist anymore, among edges between the internal nodes and leaves, the edges of group i have the highest weights. In the beginning of a phase the high-weight edges in that group have normalized weights $\frac{1}{w_i(r_i-1)}$ and the low-weight edges have weight $\frac{1}{w_i(r_i+i\cdot n^2)}$.

Hence, we have that if we sort the edges by their normalized weights, the top 3 classes of edges are, starting from the highest weight:

1. High-weight edges between internal nodes and leaves of group i.

2. Edges between the root and the internal nodes.

3. Low-weight edges between internal nodes and leaves of group i.

We will show that the phase consists of the following sub-phases.

1. First, there is some number of subphases, where each of $n-i$ internal nodes contract one incident high-weight edge.

2. Then, there is exactly one subphase, where $n-i-1$ nodes contract an incident high-weight edge and one internal node merges with the root.

3. Then, the remaining $n-i-1$ internal nodes merge with all of their group i leaves (we do not analyze the order in this subphase, as it is irrelevant).

Assume that each internal node has at least one high-weight edge in group i. Then, the algorithm will execute a Type 1 subphase: the first $n-i$ steps of the algorithm would merge exactly one high-weight edge incident to each internal node. Note that when an edge incident to an internal node v merges, the weight of v increases, and so the incident edge weights decrease. This guarantees that in the considered $n-i$ steps exactly one merge per internal node happens.

Type 1 subphases of $n-i$ steps continue as long as each each internal node has at least one high-weight group i edge in the beginning of the subphase. Each Type 1 subphase also causes the weight of each internal node to increase by 1. Clearly, since nodes are being merged into internal nodes, the ordering of edge weights incident to any internal node does not change.

At some point, in the beginning of a subphase there is an internal node that does not have any incident high-weight edge in group i. Assume that this happened after p Type 1 subphases

have completed. Thus, the size of each internal node is $w_i + p$. Since by the construction each internal node had a different number of high-weight edges in group i, there is exactly one node v with no high-weight incident edges and that node merges with the root. This is when Type 2 subphase happens. First, $n - i - 1$ internal nodes contract with a high-weight incident group i edge. At this point the edge weights are as follows. The weight of an edge between v and the root is $\frac{1}{(w_i+p)(r_i+i^2\cdot\Delta_i)}$ and the weight of a high-weight edge in group i is $\frac{1}{(w_i+p+1)(r_i-1)}$. We have that the former is larger since

$$\begin{aligned}(w_i + p)(r_i + i^2 \cdot \Delta_i) < (w_i + p + 1)(r_i - 1) &\iff\\ (w_i + p) \cdot i^2 \cdot \Delta_i < r_i - 1 &\iff\\ (n^4 + i \cdot (n + 1) + p) \cdot i^2 \cdot \Delta_i < n^8 + i \cdot n^4 - 1 &\impliedby\\ (n^4 + n \cdot (n + 1) + n) \cdot n \cdot n^2 < n^8 + i \cdot n^3 - 1.&\end{aligned}$$

Thus, the internal node with no incident high-weight edges in group i merges with the root. Observe that this is exactly the internal node which had the lowest number of high-weight edges in group i among all remaining internal nodes. This immediately implies that k_i is computed correctly, proving Item 2.

We now show that in the remaining part of the phase the $n - i - 1$ remaining internal nodes contract their incident group i edges. First, observe that the new size of the root node is

$$r_i + w_i + p + i^2 \cdot \Delta_i = \left(n^8 + i \cdot n^4 + n^4\right) + \left(i \cdot (n + 1) + i^2 \cdot \Delta_i + p\right) = r_{i+1} + (i + 1)^2\Delta_{i+1}$$

for some $\Delta_{i+1} \in [0, n + 1]$. Note that we use the fact that both Δ_i and p are upper bounded by $n + 1$, which implies $i \cdot (n + 1) + i^2 \cdot \Delta_i + p \le (i + 1)^2(n + 1)$. This proves Item 1. Thus for an internal node of weight w, the weight of its edge to the root is

$$\frac{1}{w \cdot (r_{i+1} + (i + 1)^2\Delta_{i+1})} \le \frac{1}{w \cdot r_{i+1}} = \frac{1}{w \cdot (n^8 + (i + 1) \cdot n^4)}.$$

On the other hand, its low-weight edges to group i leaves have weight

$$\frac{1}{w(n^8 + i(n^4 + n^2))}.$$

As a result, in the remaining part of the current phase all internal nodes will contract all their incident group i edges. This implies Item 4. Thus, the size of each internal node within the phase increases to $w_i + (n + 1) = n^4 + i \cdot (n + 1) + n + 1 = n^4 + (i + 1)(n + 1)$, as required. This proves Item 3 and completes the proof. □

Finally, we now claim that, given an instance of Adaptive Minimum with input index $x \in [0, n)$ and an algorithm which can compute the solutions to Problem 2, we can compute the solution k_x to Problem 3 in logspace. To see this, note that it suffices to determine the value k_x as defined above given an algorithm for Average Linkage HAC. To see this, note that for any internal node i, we can query the Average Linkage HAC algorithm to determine which time step t_i it merged with the root. This does not directly tell us which phase i merged with the root, but for a given i we can determine if it merged in phase x by comparing t_i with t_j for all $j \in \{0, 1, \ldots, n - 1\} \setminus \{i\}$, and checking if there are exactly $x - 1$ values of t_j smaller than t_i. This clearly can be verified in

log-space. Repeating for all $i \in \{0, 1, \ldots, n-1\}$ allows us to correctly determine the identity of the internal node that merged with the root in phase x, and therefore the value of k_x, in logspace as required.

To complete the proof of the Lemma it remains to show how to drop the assumption on the node sizes being initially not all equal to 1. In order to obtain a node of size w it suffices to create a node of weight 1 and initially connect it to $w-1$ auxiliary nodes using very high weight edges. This will force the algorithm to merge all these auxiliary nodes and increase the size of that node to w. Since the auxiliary leaves are connected only to the root and internal nodes (the leaves in our construction have weight 1), the diameter of the tree does not increase. □

5 Average Linkage HAC on Paths in NC

In this section, we present an $\tilde{O}(n)$ work and $O(\mathsf{polylog}(n))$ depth algorithm for solving average linkage HAC on path graphs, provided that the aspect ratio of the input instance is bounded by $\mathsf{poly}(n)$. The aspect ratio is defined as $\mathcal{A} = W_{\max}/W_{\min}$, where $W_{\max} = \arg\max_{e \in E} w(e)$ and $W_{\min} = \arg\min_{e \in E} w(e)$ (note that this definition excludes all non-edges, which implicitly have a weight of 0). The main algorithm for this section is presented in Algorithm 3, which we first give a high-level overview of next, followed by a detailed description.

Input: A path graph $G = (V, E, w)$
Function `PathHAC`(x, y, w, S):
  Compute buckets $B_1, B_2, \ldots, B_T$ such that B_i contains edges with weights in $((2/3)^t w_{\max}, (2/3)^{t-1} w_{\max}]$.
  for $t = 1$ to T do
    Let $\mathcal{C} \leftarrow$ Nearest-neighbor chains on the induced graph $G[B_t]$.
    parallel for each $C \in \mathcal{C}$:
      `ProcessChain`(C)

Input: A chain $C = (c_1, c_2, \ldots, c_N)$ such that c_{i-1} is the nearest neighbor of c_i and (c_1, c_2) is a reciprocal pair.
Function `ProcessChain`(C, t):
  Split the chain C at all indices i such that $S(c_i) < 2S(c_{i-1})$, except $i = 2$.
  We get a set of contiguous subchains $C_1, C_2, \ldots, C_K$. Subchain C_j falls into one of two categories: (A) the nearest clusters in C_{j-1} and C_j merge, or (B) no two clusters from C_{j-1} and C_j merge.
  Find the merge categories for all subchains to obtain the modified subchains $C'_1, C'_2, \ldots, C'_K$.
  parallel for each $j = 1$ to k:
    Run average linkage HAC on C'_j until no edge remains with weight within the threshold of B_t.

Algorithm 3: Average linkage HAC on paths; `ProcessChain` assumes that the input chain has a certain structure. However, this assumption can be removed easily (see Section 5)

In average linkage HAC, the weight (i.e., similarity) of edges monotonically decreases over time.

Thus, our idea is to partition the edges into buckets where the edges in any bucket have the same similarity, up to constant factors. Next, we process these buckets in phases, from the highest similarity bucket to the lowest. In each phase, we perform a modified version of the classic nearest-neighbor chain algorithm (Algorithm 4), wherein we compute the nearest-neighbor chains for the graph induced on the edges in that bucket, and process each chain independently. We note that when we use the terminology nearest neighbor of a vertex in what follows, we refer to the neighbor along the highest weight edge incident to the vertex.

Initially, each cluster is a singleton, and we might end up with $\Omega(n)$ sequential dependencies to resolve. However, we observe that in this special case when the size of every cluster is equal, starting with the reciprocal pair, every alternate edge in this chain can be merged independently, and the rest of the edges will be moved to a later bucket. We can compute the edges that will be merged easily via a simple prefix-sum routine [7]. However, when the cluster sizes are arbitrary, this observation no longer holds. Nonetheless, we show that we can partition each chain further into $O(\log n)$-sized subchains such that, even though the dependencies within a subchain must be resolved sequentially, the dependencies across subchains can be resolved in parallel using a similar application of prefix-sum. In this section we show (1) that our parallel algorithm is highly efficient (it runs near-linear time in the number of nodes) and runs in poly-logarithmic depth and (2) that our algorithm implies that the dendrogram height of a path input with polynomial aspect ratio is always poly-logarithmic.

Implementation Details and Correctness We now elaborate on certain implementation aspects of Algorithm 3. In Algorithm 3, there are two implicit assumptions made about the chains:

- Every edge in B_t is present in some chain in $\mathcal{C}$.
- Each chain can be represented as $(c_1, c_2, \ldots, c_N)$ such that c_{i-1} is the nearest neighbor of cluster c_i, and (c_1, c_2) is the reciprocal pair. Thus, the weights are in non-increasing order from left to right.

We will see later how to remove these assumptions. By the above assumptions, each chain will be a separate connected component in the graph induced on B_i, thus, allowing us to process each chain independently. In `ProcessChain`, we split each chain at all indices such that $S(c_i) < 2S(c_{i-1})$ giving us a set of subchains. This implies that for two adjacent clusters c_{j-1}, c_j in the same subchain, $S(c_j) \geq 2S_{j-1}$ (see Figure 6). Thus, the size of a subchain cannot exceed $\log n$, and we can afford to process these subchains sequentially, while still obtaining low depth overall.

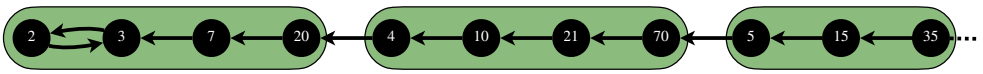


Figure 6: An example describing the partition of a chain into subchains based on cluster sizes.

We now prove certain key properties about subchains. Let u_0, u_1, u_2 denote three contiguous clusters in a chain such that u_0 is the last cluster in its subchain, and u_1 is the first cluster of its subchain. u_2 may or may not belong to the same subchain as u_1.

Lemma 3. If cluster u_1 first merges with the cluster (containing) u_0, then the weight of the edge (u_1, u_2) reduces to at most 2/3 times its previous weight.

Proof. Let $x = w(e)/(S(u_1)S(u_2))$ denote the weight of edge $e = (u_1, u_2)$. If cluster u_1 merges with a cluster in its preceding subchain, the merged cluster will be of size at least $S(u_0) + S(u_1)$. Let x' denote the new weight of edge e after this merge. Then,

$$\begin{aligned} x' \leq \frac{w(e)}{(S(u_0)+S(u_1))S(u_2)} &\implies \frac{1}{x'} \geq \frac{1}{x} + \frac{S(u_0)S(u_2)}{w(e)} \\ &\implies \frac{1}{x'} \geq \frac{1}{x} + \frac{S(u_1)S(u_2)}{2w(e)}, \qquad \text{since } S(u_0) \geq S(u_1)/2 \\ &\implies \frac{1}{x'} \geq \frac{3}{2x}, \end{aligned}$$

Therefore, $x' \leq 2x/3$. □

Lemma 4. If cluster u_1 first merges with the cluster (containing) u_2, then the weight of the edge (u_0, u_1) reduces to at most $1/3$ times its previous weight.

The proof follows by a similar argument as in Lemma 3. By Lemmas 3 and 4, each subchain falls into one of the following categories:

(A) The first cluster of the subchain merges with a cluster in the preceding subchain (if exists).

(B) The first cluster of the subchain doesn't merge with a cluster from the preceding subchain.

In either case, one of the edges incident on the first cluster of the subchain moves to a later bucket after it merges, disconnecting the two subchains in the induced graph. Thus, if a subchain falls into category (A), we can move its first cluster to the preceding subchain; if it falls into category (B), we make no changes. By this process, we obtained the modified subchains $C'_1, C'_2, \ldots, C'_K$, and by the above argument we can run average linkage HAC independently on them. Note that we do not run HAC to completion at these subchains, rather until we have edges with weight within the threshold defined by its bucket.

Note that the first subchain of a chain will always fall into category (B) since it contains a reciprocal pair which is guaranteed to merge. Running HAC on this subchain (starting with merging the reciprocal pair) determines whether the next subchain falls into the merge category (A) or (B), which in turn determines the merge category for the following subchain, and so on. We now show that we can propagate this merge behavior across subchains with the help of a simple prefix-sum routine in $\mathsf{poly}(\log n)$ depth.

Lemma 5. Given a chain of size N partitioned into subchains $C_1, C_2, \ldots, C_K$, we can determine the merge category ((A) or (B)) for each subchain in $\tilde{O}(N)$ work and $\mathsf{poly}(\log n)$ depth.

Proof. For subchain C_j, let $f(j, A)$ denote the resulting merge category of C_{j+1} if C_j belongs to category (A). Define $f(j, B)$ similarly. Firstly, recall that the category of C_1 is determined (it is always category (B)). If $f(j, A) = f(j, B)$, for some j, this implies that the category of C_{j+1} is determined regardless of the category of C_j. Thus, we can split the chain at subchain C_{j+1} and deal with the two chains obtained, independently. We now assume that $f(j, A) \neq f(j, B)$ for all j.

We create an array A of size K such that,

$$A[j] = \begin{cases} 0 & \text{if } j = 1 \text{ and } f(1, B) = A, \\ 1 & \text{if } j = 1 \text{ and } f(1, B) = B, \\ 0 & \text{if } j > 1 \text{ and } f(j, A) = A, \\ 1 & \text{if } j > 1 \text{ and } f(j, A) = B. \end{cases}$$

Let $B = \textsf{prefix-sum}(A, \texttt{XOR}, 0)$, i.e., $B[1] = 0$ and $B[i] = A_1 \oplus A_2 \oplus \ldots A_{i-1}$. In other words, we compute the prefix sum of the array A with respect to the XOR function, using 0 as the left identity element. Then, we claim that the merge category of subchain C_j is (A) if $B[j] = 0$, otherwise it is (B).

To see this, consider subchains such that $f(j, A) = A$. These subchains propagate the same category (as them) to the next subchain, which can be viewed as an XOR operation with a 0. Similarly, subchains that have $f(j, A) = B$ propagate the opposite category (as them) to the next subchain, which can be viewed as an XOR with 1. $A[1]$ denotes the category of subchain C_2 that is determined by C_1. Thus, computing the prefix-sum correctly computes the categories for each subchain.

The values of $f(j, A)$ (and $f(j, B)$) can be computed by simply simulating average linkage HAC sequentially at each subchain, assuming the category (A) (and (B)) at that subchain. Since the subchain sizes are at most $\log n$, the work incurred to solve HAC on a subchain of size r will be $O(r \log \log n)$.

To see this, consider running Algorithm 5 on the chain, which we claim runs in $O(N \log \log n)$ work (and depth). This is because the heap-based algorithm for HAC (described in more detail in Section 6) on a path containing $r \leq \log n$ elements operates over a heap containing at most $\log n$ elements, and thus each of the at most r merge operations require $O(\log \log n)$ time and only require updating the two neighboring edges incident to the merged edge. Since the algorithm used on each subchain is sequential, the depth at each subchain will also be $O(r \log \log n) \in O(\log n \log \log n)$. The prefix-sum operation runs in $O(K)$ work and $O(\log K)$ depth.

Thus, the overall work for a chain containing N elements will be $O(N \log \log n)$ and depth will be $O(\log n \log \log n)$. □

Since the edges are processed in non-increasing order of weight (due to bucketing), and Lemmas 3 to 5 correctly computes the subchains and proves the correctness of processing these subchains independently, the overall correctness of the algorithm follows directly.

Work and Depth Bounds We will now argue the work and depth bounds of the overall algorithm. Observe that, since the minimum possible weight is at least $W_{\min}/n^2$, the total number of phases will be $T \in O(\log \mathcal{A} + \log n) \in O(\log n)$. In each phase, we can compute the nearest neighbor chains in $O(n)$ work and $O(\log n)$ depth; the subchains can also be computed with similar work-depth bounds. By Lemma 5, the total work incurred in finding the merge categories for each subchain and running average linkage HAC on these subchains, across all chains, in the worst case will be $O(n \log \log n)$ work and $O(\log n \log \log n)$ depth. Thus, the overall work of the algorithm is $O(n \log n \log \log n)$ and depth is $O(\log^2 n \log \log n)$.

Thus, we have the following theorem about the work and depth of the algorithm:

Theorem 4. Average linkage HAC on paths is in NC. In particular, there is an algorithm for average linkage HAC that runs in $O(\log^2 n \log \log n)$ depth with $O(n \log n \log \log n)$ work.

Resolving assumptions about chain structure We will now address the assumptions made about the structure of chains in the algorithm. When the nearest neighbor chains are computed, the structure of the chains can be represented as

$$(c'_{N'} \ldots, c'_3, c'_2, c'_1, c_1, c_2, c_3, \ldots, c_N),$$

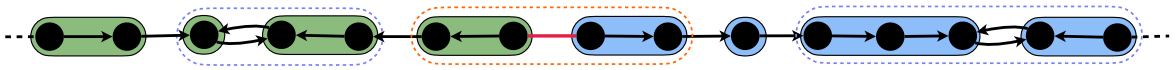


Figure 7: An example illustrating the general (possible) structure of the nearest neighbor chains computed in each phase of Algorithm 3. The red edge here is an example of an edge that is not present in any chain. The dotted lines represents the various subchain merges to address the assumptions made about the nearest neighbor chain structure in Algorithm 3.

where (c'_1, c_1) is the reciprocal pair, c'_{i-1} is the nearest neighbor of c'_i, and c_{i-1} is the nearest neighbor of c_i. We can extend the idea of partitioning these chains into subchains as follows: partition the chains $(c_1, c_2, \ldots, c_N)$ and $(c'_1, c'_2, \ldots, c'_{N'})$, independently, into subchains using the same criterion as before. Now, merge the subchains containing c_1 and c'_1 resulting in a subchain of length at most $2 \log n$ (see Figure 7). Then, independently propagate the merge behavior across these two sets of subchains, with the subchain containing (c_1, c'_1) as the first subchain.

Finally, when we compute the nearest neighbor chains, it is possible for some edges in the bucket to not be present in any chain. However, these edges can occur only between the last vertices of two chains (see Figure 7). Thus, after finding the merge categories for the subchains containing these vertices (in their respective chains), we can merge these two subchains along with the edge (whose size will be at most $4 \log n$ in the worst case). It is not hard to extend the correctness and running time arguments when these modifications are incorporated.

Height of the dendrogram is $O(\log^2 n)$ An interesting consequence of our algorithm is a constructive proof that the height of the dendrogram obtained when we run average linkage HAC on paths with polynomial aspect ratio is $O(\log^2 n)$.

Theorem 9. Average linkage HAC on path graphs with $\mathsf{poly}(n)$ aspect-ratio returns a dendrogram with height at most $O(\log^2 n)$.

Proof. Starting with phase 1, the algorithm runs average linkage HAC (partially) on independent $O(\log n)$-sized contiguous portions (i.e., subchains) of the path. Thus, each subchain will correspond to independent parts in the output dendrogram. Since there are $O(\log n)$ phases and the max-height generated in each phase is at most $O(\log n)$, we get an overall bound of $O(\log^2 n)$ for the height in the worst case. □

6 $O(mh \log n)$ Upper Bound

In this section we show that we can solve graph-based average linkage HAC in $\tilde{O}(mh)$ time where h is the height of the output dendrogram. Thus, when $mh = o(n^{3/2})$ this algorithm improves over the lower bound from Section 3. Interestingly, we give a simple analysis showing that either of the two classic approaches to HAC—the nearest-neighbor chain or heap-based algorithms—achieves this bound. We give the pseudocode for the nearest-neighbor chain algorithm in Algorithm 4 and the heap-based algorithm in Algorithm 5. In a nutshell, in both algorithms initially all vertices start in their own cluster, and are active. As the algorithms proceed, they find edges to merge that the exact greedy HAC algorithm (Algorithm 1) will make and merge them, marking one of the

```
Input: G = (V, E, w)
Function NearestNeighborChain(G):
    C ← clustering where each v ∈ V is in a separate cluster and is active.
    for each cluster v ∈ V do
        if v is active then
            Initialize a stack S initially containing only v
            while S is not empty do
                t ← Top(S).
                (t, b, w_{t,b}) ← BestEdge(t).
                if b is already on S then
                    Pop(S).
                    Merge(t, Top(S)).   // t is marked as inactive; Top(S) remains active.
                    Pop(S).
                else
                    Push(S, b).
```

Algorithm 4: The nearest-neighbor chain algorithm for HAC.

endpoints as inactive. This process continues until no further edges between active clusters remain. We discuss the algorithms individually in more detail in what follows.

We start by discussing some data structures and details in how the algorithms carry out merges.

Neighborhood representation The neighbors of each vertex are stored in a max-heap supporting standard operations (e.g., find-min, delete-min, insert, decrease-key). The priority of a neighbor is the average linkage weight of the edge to the neighbor. At the start of the algorithm, the heaps are initialized with the initial neighbors of each vertex. Let $N(A)$ represent the neighbors of a cluster A. For concreteness in what follows, we use Fibonacci heaps [14, 19].

Merging clusters Next, we discuss how to implement the merge step in both algorithms, which merges two clusters A and B, and their corresponding heaps (i.e., merging $N(A)$ and $N(B)$ and updating the average linkage weights of all edges affected by the merge). We maintain the invariant that the heaps always store the correct average linkage weights of all edges. We first compute $\mathcal{I}(A, B) = N(A) \cap N(B)$, which can be done by sorting both sets and merging the sorted arrays. For each $C \in \mathcal{I}(A, B)$, without loss of generality, we remove the reference to A in the heap and update the priority of B to reflect the new average linkage weight from C to $A \cup B$ using decrease-key. Similarly, we update each $C \in N(A) \setminus \mathcal{I}(A, B)$ to reference B instead of A. Finally, for each $C \in N(A) \cup N(B)$ we update the priority of the edge (C, B) to reflect the new size of B.

At the end of the merge: (1) $N(B)$ contains $N(A) \cup N(B) \setminus \{A, B\}$; (2) all edges in the updated $N(B)$ have their average linkage weight correctly set; and (3) after the update all $C \in N(B)$ no longer reference A, but point to B and have their average linkage weights correctly set. The cost of this merge procedure is dominated by the cost of sorting $N(A)$ and $N(B)$; plugging in the cost bounds for any reasonable heap data structure (e.g., a Fibonacci heap) yields the following lemma:

Lemma 6. Merging two clusters A, B with $N(A) + N(B) = T$ can be done in $O(T \log n)$ time.

Nearest-Neighbor Chain Analysis As we run the nearest-neighbor chain algorithm (Algorithm 4), the algorithm only requires two non-trivial operations beyond basic data structures (e.g., stacks and heaps):[2]

1. BestEdge(C): fetch the highest similarity edge incident to a cluster C.
2. Merge(A, B): merge the clusters A, B into a cluster $A \cup B$ (represented, without loss of generality by B). Thus after the merge, A is inactive, and B remains active.

Using the data structures above, we can implement both operations very quickly. In particular, Lemma 6 gives the time for merging two clusters, and BestEdge(C) can be implemented in $O(1)$ time using the find-min operation.

After merging two clusters that are reciprocal "best" neighbors, the algorithm back-tracks to the third vertex in the chain and restarts the process, using calls to BestEdge. The cost of these calls can be charged to the merges, resulting in a total of $O(n)$ time for the $n-1$ merges. The time that still must be accounted for is that of Merge.

Claim 2. The total amount time spent to perform all merges made by the nearest-neighbor chain algorithm is $O(mh \log n)$.

Proof. Split each edge (u, v) into two directed edges (u, v) and (v, u), since the edge will be present initially in both $N(u)$ and $N(v)$ (and the intermediary clusters they form until they are merged).

Consider a directed edge (u, v), and the merges it experiences as it flows along the path from the initial cluster containing it (u) to the cluster where it is finally merged. The dendrogram that the algorithm constructs has height h by definition, and thus the length of this path is at most h. In each node on the path, the edge contributes a cost of $O(\log n)$ to the merge cost in Lemma 6. Thus, the cost of this edge across the entire path is at most $O(h \log n)$.

Applying this charging argument for all $2m$ directed edges, the total cost over all merges is $O(mh \log n)$. □

Putting together the above yields the following theorem.

Theorem 5. There is an implementation of the nearest-neighbor chain algorithm for average linkage HAC that runs in $O(m \cdot h \log n)$ time where h is the height of the output dendrogram.

Heap-Based Algorithm We note it is easy to extend the argument above to obtain a similar bound for the heap-based algorithm (pseudocode shown in Algorithm 5). In particular, beyond the cost for merging, which is the same in the nearest-neighbor chain algorithm and the heap-based algorithm (captured by Claim 2), the only extra work incurred by the heap-based algorithm is due to edges unsuccessfully extracted from the heap (Line 7 of Algorithm 5). The number of such unsuccessful edges is at most m, and each such edge incurs $O(\log n)$ work due to an extra call to BestEdge(u), for a total of $O(m \log n)$ extra work. Thus the total work of the heap-based algorithm is also $O(mh \log n)$.

Theorem 10. There is an implementation of the heap-based algorithm for average linkage HAC that runs in $O(mh \log n)$ time where h is the height of the output dendrogram.

[2] In fact the same two primitives are also the only non-trivial ones used for the heap-based algorithm (Algorithm 5) and thus we fully specify the details for this algorithm as well.

```
Input: G = (V, E, w)
Function HeapBasedHAC(G):
    C ← clustering where each v ∈ V is in a separate cluster and is active.
    Let H be a max-heap storing the highest-weight edge incident to each active cluster in
     the graph
    while |H| > 1 do
        Let e = (u, v, w_{u,v}) be the max weight edge in H.
        Delete e from H.
        if v is inactive then
            Let e' = (u, v', w_{u,v'}) = BestEdge(u).
            Insert e' into H.
        else
            x = Merge(u, v).   // u is marked as inactive; v remains active.
            Let e' = (x, y, w_{x,y}) = BestEdge(x).
            Insert e' into H.
```

Algorithm 5: The heap-based algorithm for HAC.

7 Conclusion

In this paper, we studied the parallel and sequential complexity of hierarchical graph clustering. We gave new classic and fine-grained reductions for Hierarchical Agglomerative Clustering (HAC) under the average linkage measure that likely rule out efficient algorithms for exact average linkage, parallel or otherwise. We also showed that such impossibility results can be circumvented if the output dendrogram has low height or is a path. An interesting question is whether such structure can be leveraged for other variants of interest of average linkage HAC: for example, can we can obtain dynamic algorithms for HAC that are also parameterized by the height?

References

[1] Amir Abboud, Vincent Cohen-Addad, and Hussein Houdrouge. Subquadratic high-dimensional hierarchical clustering. 2019.

[2] Amir Abboud, Nick Fischer, and Yarin Shechter. Faster combinatorial k-clique algorithms. arXiv preprint arXiv:2401.13502, 2024.

[3] Josh Alman and Virginia Vassilevska Williams. A refined laser method and faster matrix multiplication. In Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 522–539. SIAM, 2021.

[4] N. S. Arora, R. D. Blumofe, and C. G. Plaxton. Thread scheduling for multiprogrammed multiprocessors. ACM Transactions on Computer Systems, 34(2), Apr 2001.

[5] MohammadHossein Bateni, Soheil Behnezhad, Mahsa Derakhshan, MohammadTaghi Hajiaghayi, Raimondas Kiveris, Silvio Lattanzi, and Vahab Mirrokni. Affinity clustering: Hierarchical clustering at scale. In Annual Conference on Neural Information Processing Systems (NeurIPS), pages 6864–6874, 2017.

[6] J-P Benzécri. Construction d'une classification ascendante hiérarchique par la recherche en chaîne des voisins réciproques. Cahiers de l'analyse des données, 7(2):209–218, 1982.

[7] Guy E Blelloch. Scans as primitive parallel operations. IEEE Transactions on computers, 38(11):1526–1538, 1989.

[8] Guy E. Blelloch, Jeremy T. Fineman, Yan Gu, and Yihan Sun. Optimal parallel algorithms in the binary-forking model. In ACM Symposium on Parallelism in Algorithms and Architectures (SPAA), 2020.

[9] Robert D. Blumofe and Charles E. Leiserson. Space-efficient scheduling of multithreaded computations. 27(1), 1998.

[10] Mélanie Boly, Vincent Perlbarg, Guillaume Marrelec, Manuel Schabus, Steven Laureys, Julien Doyon, Mélanie Pélégrini-Issac, Pierre Maquet, and Habib Benali. Hierarchical clustering of brain activity during human nonrapid eye movement sleep. Proceedings of the National Academy of Sciences, 109(15):5856–5861, 2012.

[11] Vincent Cohen-Addad, Varun Kanade, Frederik Mallmann-Trenn, and Claire Mathieu. Hierarchical clustering: Objective functions and algorithms. Journal of the ACM (JACM), 66(4), 2019.

[12] Stephen A Cook, Yuval Filmus, and Dai Tri Man Le. The complexity of the comparator circuit value problem. ACM Transactions on Computation Theory (TOCT), 6(4):1–44, 2014.

[13] Don Coppersmith and Shmuel Winograd. On the asymptotic complexity of matrix multiplication. SIAM Journal on Computing, 11(3):472–492, 1982.

[14] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms (3rd edition). MIT Press, 2009.

[15] Laxman Dhulipala, David Eisenstat, Jakub Lacki, Vahab Mirrokni, and Jessica Shi. Hierarchical agglomerative graph clustering in nearly-linear time. In International Conference on Machine Learning (ICML), pages 2676–2686, 2021.

[16] Laxman Dhulipala, David Eisenstat, Jakub Lacki, Vahab Mirrokni, and Jessica Shi. Hierarchical agglomerative graph clustering in poly-logarithmic depth. Annual Conference on Neural Information Processing Systems (NeurIPS), 35:22925–22940, 2022.

[17] Laxman Dhulipala, Jakub Łącki, Jason Lee, and Vahab Mirrokni. Terahac: Hierarchical agglomerative clustering of trillion-edge graphs. Proceedings of the ACM on Management of Data, 1(3):1–27, 2023.

[18] Michael B Eisen, Paul T Spellman, Patrick O Brown, and David Botstein. Cluster analysis and display of genome-wide expression patterns. Proceedings of the National Academy of Sciences, 95(25):14863–14868, 1998.

[19] Michael L Fredman and Robert Endre Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. Journal of the ACM (JACM), 34(3):596–615, 1987.

[20] Raymond Greenlaw and Sanpawat Kantabutra. On the parallel complexity of hierarchical clustering and cc-complete problems. Complexity, 14(2):18–28, 2008.

[21] Guan-Jie Hua, Che-Lun Hung, Chun-Yuan Lin, Fu-Che Wu, Yu-Wei Chan, and Chuan Yi Tang. MGUPGMA: a fast UPGMA algorithm with multiple graphics processing units using NCCL. Evolutionary Bioinformatics, 13:1176934317734220, 2017.

[22] J Juan. Programme de classification hiérarchique par l'algorithme de la recherche en chaîne des voisins réciproques. Cahiers de l'analyse des données, 7(2):219–225, 1982.

[23] Benjamin King. Step-wise clustering procedures. Journal of the American Statistical Association, 62(317):86–101, 1967.

[24] Ari Kobren, Nicholas Monath, Akshay Krishnamurthy, and Andrew McCallum. A hierarchical algorithm for extreme clustering. In International Conference on Knowledge Discovery and Data Mining (KDD), pages 255–264, 2017.

[25] Godfrey N Lance and William Thomas Williams. A general theory of classificatory sorting strategies: 1. hierarchical systems. The computer journal, 9(4):373–380, 1967.

[26] Joshua Lau and Angus Ritossa. Algorithms and hardness for multidimensional range updates and queries. In Innovations in Theoretical Computer Science Conference (ITCS), 2021.

[27] François Le Gall. Faster algorithms for rectangular matrix multiplication. In Symposium on Foundations of Computer Science (FOCS), pages 514–523. IEEE, 2012.

[28] Christopher D Manning, Prabhakar Raghavan, and Hinrich Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008.

[29] Ernst W Mayr and Ashok Subramanian. The complexity of circuit value and network stability. Journal of Computer and System Sciences, 44(2):302–323, 1992.

[30] Nicholas Monath, Kumar Avinava Dubey, Guru Guruganesh, Manzil Zaheer, Amr Ahmed, Andrew McCallum, Gokhan Mergen, Marc Najork, Mert Terzihan, Bryon Tjanaka, et al. Scalable hierarchical agglomerative clustering. In International Conference on Knowledge Discovery and Data Mining (KDD), pages 1245–1255, 2021.

[31] Benjamin Moseley and Joshua R. Wang. Approximation bounds for hierarchical clustering: Average linkage, bisecting k-means, and local search. In Annual Conference on Neural Information Processing Systems (NeurIPS), pages 3094–3103, 2017.

[32] Benjamin Moseley and Joshua R Wang. Approximation bounds for hierarchical clustering: Average linkage, bisecting k-means, and local search. Journal of Machine Learning Research, 24(1):1–36, 2023.

[33] Daniel Müllner. Modern hierarchical, agglomerative clustering algorithms. arXiv preprint arXiv:1109.2378, 2011.

[34] Fionn Murtagh and Pedro Contreras. Algorithms for hierarchical clustering: an overview. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 2(1):86–97, 2012.

[35] Fionn Murtagh and Pedro Contreras. Algorithms for hierarchical clustering: an overview, ii. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 7(6):e1219, 2017.

[36] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. the Journal of machine Learning research, 12:2825–2830, 2011.

[37] JM Shearer and Michael A Wolfe. Alglib, a simple symbol-manipulation package. Communications of the ACM, 28(8):820–825, 1985.

[38] Peter Henry Andrews Sneath. The principles and practice of numerical classification. Numerical taxonomy, 573, 1973.

[39] Ashok Subramanian. A new approach to stable matching problems. Stanford University, 1989.

[40] Tom Tseng, Laxman Dhulipala, and Julian Shun. Parallel batch-dynamic minimum spanning forest and the efficiency of dynamic agglomerative graph clustering. In ACM Symposium on Parallelism in Algorithms and Architectures (SPAA), page 233–245, 2022.

[41] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. Scipy 1.0: fundamental algorithms for scientific computing in python. Nature methods, 17(3):261–272, 2020.

[42] Virginia Vassilevska Williams. Multiplying matrices faster than coppersmith-winograd. In Annual ACM Symposium on Theory of Computing (STOC), pages 887–898, 2012.

[43] Virginia Vassilevska Williams and Ryan Williams. Subcubic equivalences between path, matrix and triangle problems. In Symposium on Foundations of Computer Science (FOCS), pages 645–654, 2010.

[44] Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. New bounds for matrix multiplication: from alpha to omega. In Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 3792–3835. SIAM, 2024.

[45] Ying Zhao and George Karypis. Evaluation of hierarchical clustering algorithms for document datasets. In Conference on Information and Knowledge Management (CIKM), pages 515–524, 2002.